Transformers

Natural Language Processing

Daniel Ortiz Martínez

Table of Contents

1. Introduction

2. Transformer Architectures

3. Transformer Implementations

4. Miscellaneous Topics

Introduction

Transformers

- The transformer architecture (Vaswani et al. 2017) is based on the encoder-decoder framework
- Main idea: use attention mechanisms only
- Both the encoder and the decoder parts use stacked attention layers
- The transformer architecture is the foundation of current AI revolution

Large Language Models

- A language model (LM) defines a probability distribution over sequences of words
- LMs generate probabilities based on a set of parameters
- State-of-the-art LMs are based on the transformer architecture
- Large language models (LLMs) are LMs with >1B parameters

Transfer Learning

- Data scarcity may make difficult to apply machine learning (ML) to a certain task
- However, there may be a related task with plenty of data
- Transfer learning makes possible to transfer knowledge from the resource-rich task to the under-resourced one
- Transfer learning deemed as the next driver of ML commercial success

Transfer Learning

- Multiple approaches, but the main one used in LLMs is pre-training
- With model pre-training, a model trained for a general task is *fine-tuned* for a more specific or *downstream* task
- Pre-training features
  - Large and diverse data requirements
  - Computationally expensive
  - Useful in different applications
- Fine-tuning features
  - Smaller data requirements from a specific domain
  - Computationally cheaper
  - Useful in a specific application

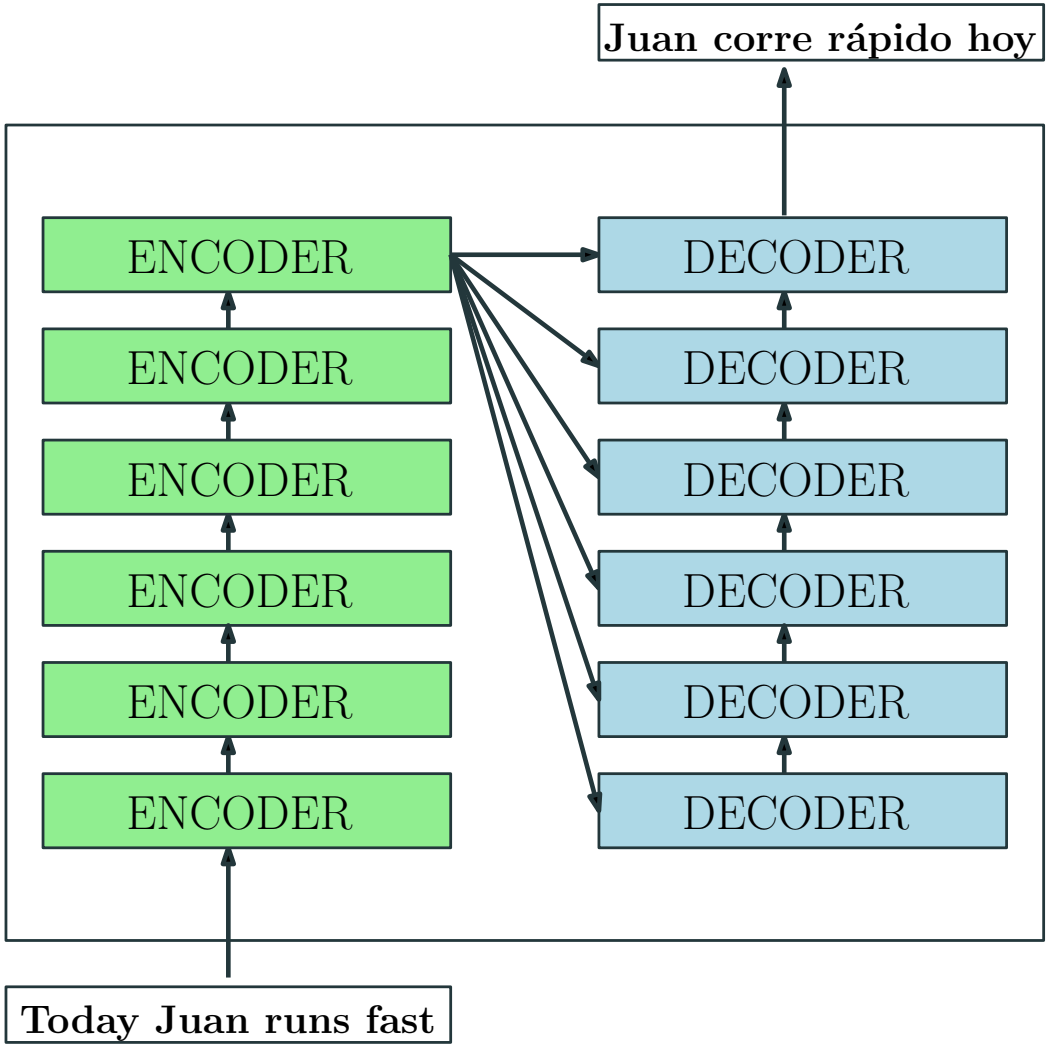
Zero-Shot and Few-Shot Learning

- Zero-shot and few-shot learning have gained very strong attention in connection with the development of LLMs
- **Zero-shot learning**: a classifier/learner is given samples from classes not seen during training and should predict the class they belong to
- **Few-shot learning**: a classifier/learner should predict the class of a set of samples for which only a few labeled training samples were available
- The ability of a model to perform zero-shot or few-shot learning is key for its widespread application due to supervised data scarcity

Transformer Architectures

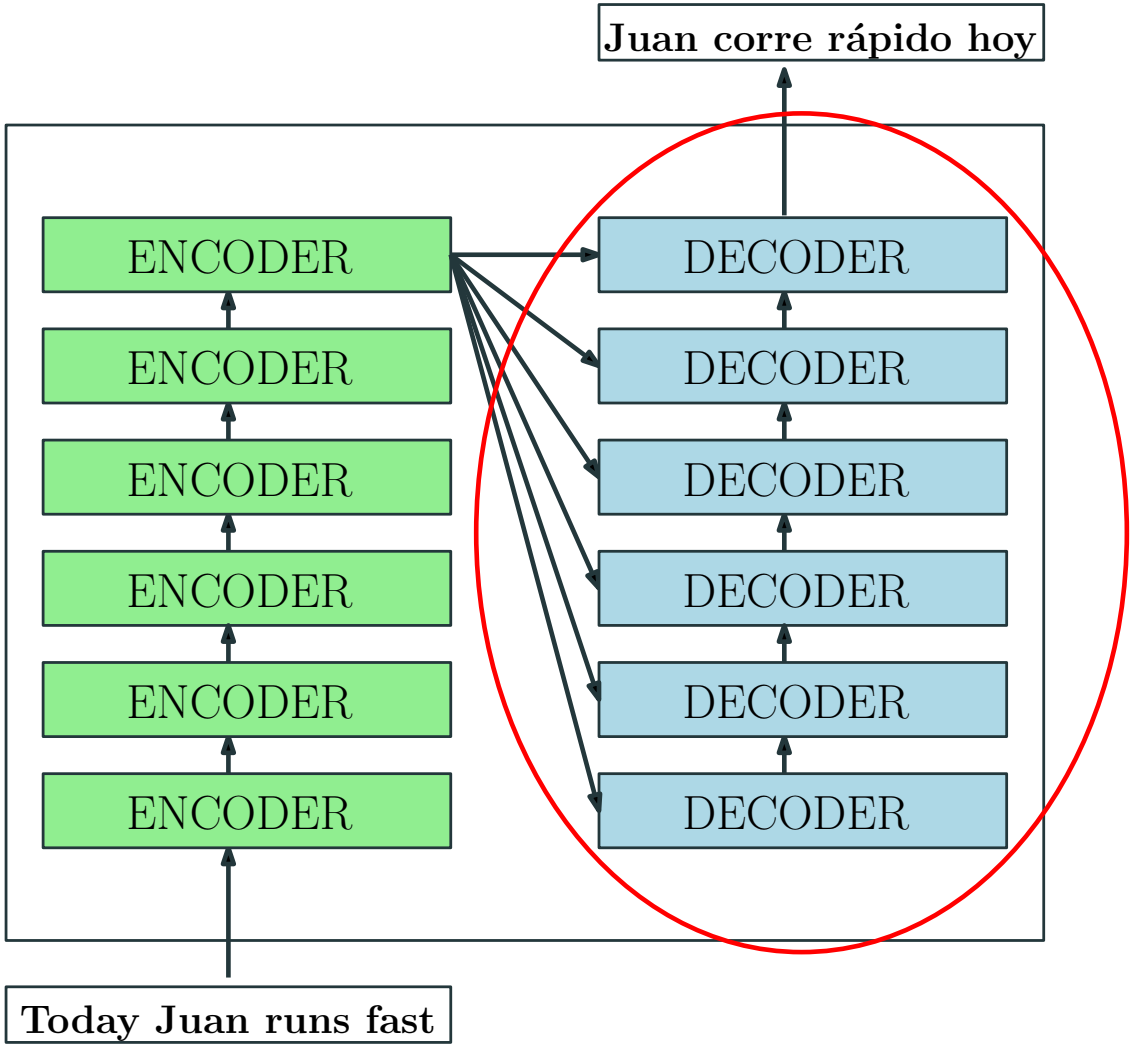
The Transformer Architecture (or Sequence-to-Sequence)

(Vaswani et al. 2017)



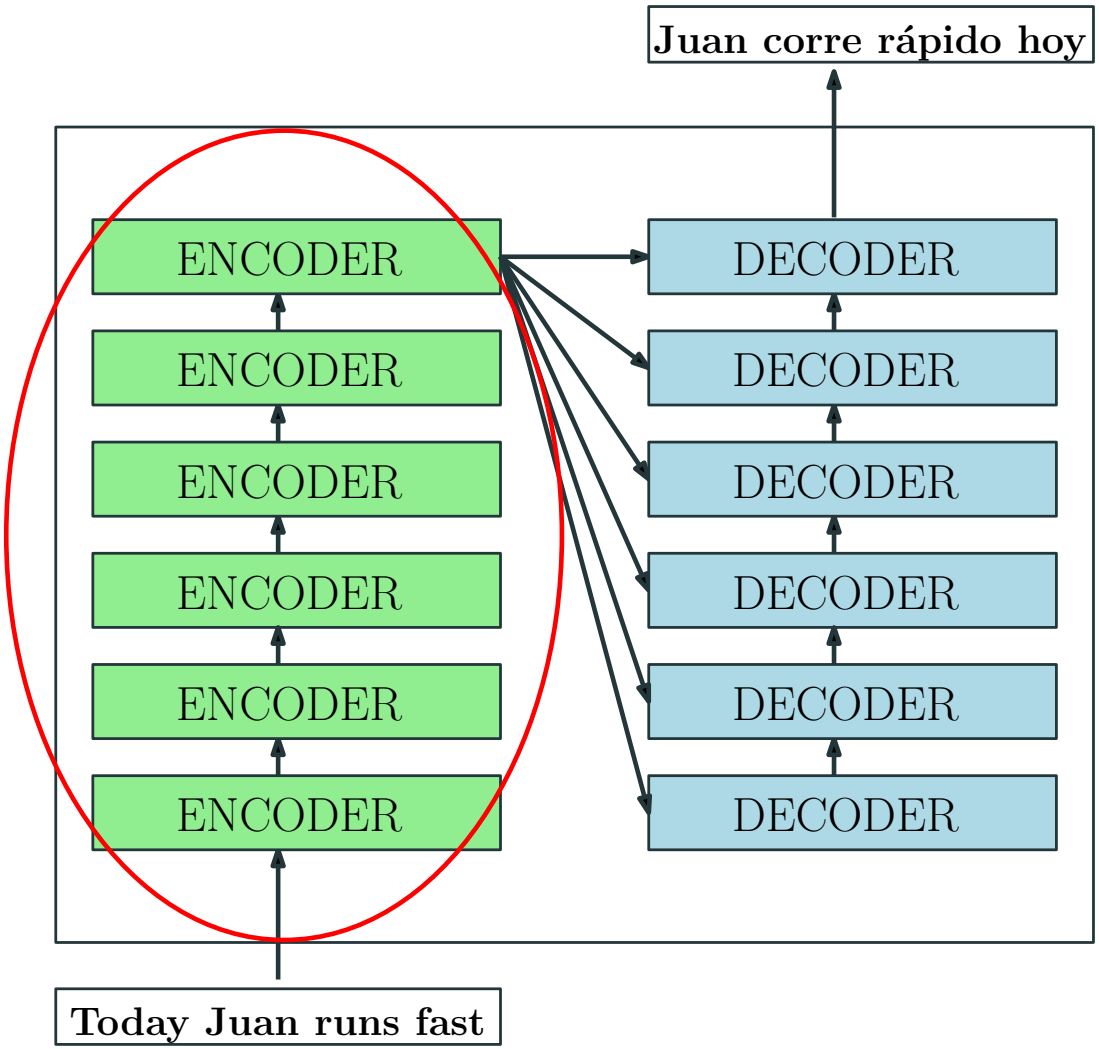
GPT

(Radford, Narasimhan, et al. 2018)



BERT

(Devlin et al. 2019)



Choosing the Right Transformer Architecture

- Choosing one architecture or another is tightly related to the ML task we need to solve
- BERT architecture: useful for classification tasks, less appropriate for text generation tasks
- GPT architecture: useful for text generation
- Whole transformer architecture: useful to solve sequence-to-sequence tasks (e.g. machine translation), can also be applied for classification

Transformer Implementations

Tokenization

- LLMs do not process raw text, but sequences of *tokens*
- A token can be a word, a subword unit, or a character
- The vocabulary is the set of all possible tokens; its size impacts model expressiveness and efficiency
- Popular tokenization algorithms:
  - **Byte-Pair Encoding** (BPE) (Sennrich et al. 2016): iteratively merges the most frequent pair of symbols; used in GPT-2, RoBERTa
  - **WordPiece**: similar to BPE but uses likelihood instead of frequency; used in BERT
  - **SentencePiece**: language-agnostic, operates on raw Unicode; used in T5, LLaMA
- Tokenization affects model performance

BERT

- BERT (Devlin et al. 2019) uses the encoder part of the transformer
- Tasks solved by the pre-trained model:
  - **Masked language modelling** (MLM): predicts masked words in a text
  - **Next sentence prediction** (NSP): given two sentences, predict whether they are consecutive or not
- Tasks solved by fine-tuned models:
  - Text classification
  - Sentiment analysis
  - Question answering
  - ...
- Not that useful for text generation

RoBERTa

- RoBERTa (Liu et al. 2019) is a BERT model with some tweaks:
  - Different tokenizer encoding: BPE (Sennrich et al. 2016)
  - Training with larger batches
  - Dynamic masking: masked words vary from one epoch to another
  - The NSP objective is removed from the pre-training scheme
- Tasks solved by the pre-trained model:
  - **Masked language modelling** (MLM): predicts masked words
- Tasks solved by fine-tuned models: same as BERT
- RoBERTa is also not appropriate for text generation

Distilled BERT-like Models

- A technique called *distillation* can be used to obtain smaller BERT-like models with almost the same performance of their larger counterparts
- Distillation trains a small *student model* to mimic a larger *teacher model*
- The canonical distilled model is DistilBERT (Sanh et al. 2019)
  - trained to predict the same probabilities as the original BERT model
  - predicts masked words but no next-sentence objective
  - preserves similarity between the hidden states of the student and teacher models

SBERT

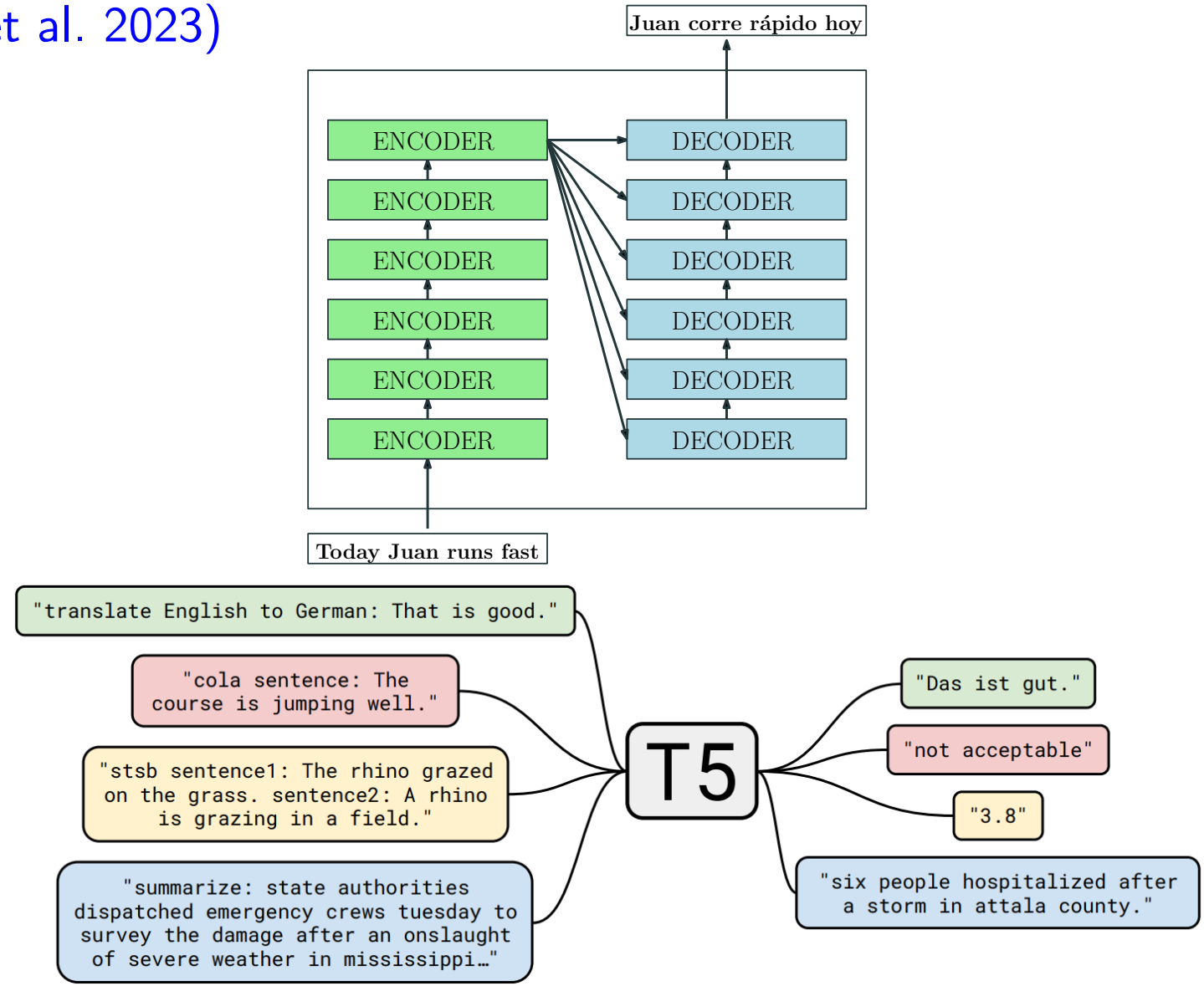
- Sentence-BERT (SBERT) (Reimers and Gurevych 2019) is a BERT model adapted for sentence embedding generation
- Conventional BERT internally generates information that can be used to obtain sentence embeddings
- However, those sentence embeddings are not of high quality
- SBERT solves this problem by means of two elements:
  - A pooling layer to the output of BERT
  - A fine-tuning process using supervised sentence pairs or triplets whose goal is to assign similar embeddings to similar sentences

BART

- BART (M. Lewis et al. 2019) follows a sequence-to-sequence architecture
- BART introduces a few network modifications with respect to the standard transformer architecture
- BART's pretrained model solves the task of reconstructing a previously corrupted or noised input
- Tasks solved by fine-tuned models: BERT-related tasks but also enables text generation tasks

T5 (Text-to-Text Transfer Transformer)

(Raffel et al. 2023)



Transformers for Text Generation

- Text generation is also called causal language modelling
- Models typically follow a GPT or a sequence-to-sequence architecture
- Two main types:
  - **Language model**: given a context predicts the next word
  - **Instruction model**: has been trained to follow instructions (instruction fine-tuning)
- Some examples:
  - OpenAI GPT-2 (Radford, Wu, et al. 2018)
  - LLaMA (Touvron et al. 2023)
  - Falcon[a]
  - Mistral[b]

[a] https://falconllm.tii.ae/

[b] https://mistral.ai/news/announcing-mistral-7b/

Transformers for Text Generation: Commercial Models

- GPT (OpenAI): dominant general-purpose model family; GPT-5 is state of the art in 2026
- Gemini (Google): strong multimodal capabilities; Gemini 2.5 Pro excels at interactive applications
- Claude (Anthropic): focus on AI safety; strong at long-document reasoning and analysis
- Mistral (Mistral AI): frontier-class performance at reduced cost; suitable for high-volume applications
- Grok (xAI): real-time data integration and current information retrieval
- DeepSeek (DeepSeek AI): competitive open-weight model developed at significantly lower cost; strong in technical reasoning

Transformers for Code

- Code can be seen as text belonging to a (non-natural) language
- There are many interesting tasks and applications related to code (code generation, code completion, code review, etc.)
- Wide range of transformer models working with code:
  - Code2Vec (Alon et al. 2018): code embedding generation
  - CodeT5 (Wang et al. 2021): T5 model, text to code, code completion and summarization
  - StarCoder (Li et al. 2023): LLM for code generation
  - WizardCoder (Luo et al. 2023): Code LLM with instruction fine-tuning
  - Also possible to apply general transformers: GPT, Gemini, LLaMA, etc.

Transformers for Biology: DNABERT

- Transformers can be useful in all those cases where we have sequential data that is categorical
- This type of data is common in biology (e.g. nucleotide or amino acid sequences)
- DNABERT (Ji et al. 2021) is a BERT-like model trained on a DNA genome reference
- Examples of downstream tasks:
  - prediction of promoters (locations where DNA transcription starts)
  - prediction of splice sites (locations involved in the maturation of RNA)
  - prediction of transcriptor factor binding sites (locations involved in the regulation of gene expression)

Transformers for Other Modalities

- Transformers cannot only be applied to textual modalities
- Transformers for images
  - Example: Visual Transformer (Dosovitskiy et al. 2021)
  - Survey: (Khan et al. 2022)
- Transformers for audio
  - Example: SpeechT5 (Ao et al. 2022)
  - Survey: (Latif et al. 2023)
- Transformers for video
  - Example: TimesSformer (Bertasius et al. 2021)
  - Survey: (Selva et al. 2023)
- Transformers can also combine modalities (multimodal)
  - Survey: (Xu et al. 2023)

Miscellaneous Topics

LLM Operation

- Operating LLMs is computationally expensive and frequently carried out with cloud computing resources
- However, there are initiatives to make LLM operation affordable in consumer hardware, even using CPUs
- A really nice collective effort is `llama.cpp`[c]
  - Supported platforms: Linux, Mac OS, Windows
  - Supported hardware: GPUs, CPUs, Hybrid GPU+CPU
  - Supported models: LLaMA, Mistral, Falcon, Starcoder, etc.
  - One interesting trick applied: quantization
  - A rich software ecosystem is being built around the package

[c] https://github.com/ggerganov/llama.cpp

LLM Training

- LLM training even more computationally expensive than LLM operation
- When fine-tuning a model, not all the weights may need to be substancially modified
- Low-Rank Adaptation (Hu et al. 2021) (LoRA) indirectly trains the dense layers of a given network by optimizing rank decomposition matrices of the dense layers' change during adaptation
  - the number of trainable parameters can be reduced by 10 000 times
  - the GPU memory requirements can be reduced by 3 times
- Quantized LoRA (Dettmers et al. 2023) (QLoRA) enables LoRA for quantized LLMs

LLM Evaluation

- LLM evaluation remains an open problem
  - What LLMs can do well?
  - Where LLMs fail?
- Two types of evaluation measures:
  - Automatic: objective, cheap, fast, typically quantitative
  - Manual: subjective, expensive, slow, quantitative or qualitative
- Benchmarking has become a popular approach, e.g. Open LLM Leaderboard

LLM Evaluation

- Some popular metrics:
  - ARC: grade-school level, multiple-choice science questions
  - HellaSwag: evaluates a model's common sense reasoning.
  - MMLU: covers a variety of academic topics across a variety of levels
  - TruthfulQA: model's ability to provide truthful and factual responses
- There are evaluation frameworks for LLMs, e.g. DeepEval
- Two survey papers on LLM evaluation: (Chang et al. 2023; Guo et al. 2023)

LLM Alignment: RLHF

- A pre-trained LLM predicts text but is not necessarily helpful, harmless, or honest
- **Reinforcement Learning from Human Feedback** (Ouyang et al. 2022) (RLHF) is the dominant technique for aligning LLMs with human preferences
- Three stages:
  - **Supervised fine-tuning** (SFT): fine-tune on human-written demonstrations
  - **Reward model training**: train a model to score outputs based on human preference comparisons
  - **RL fine-tuning**: use the reward model to further optimize the LLM via Proximal Policy Optimization (PPO)
- Used to train most modern instruction-following models

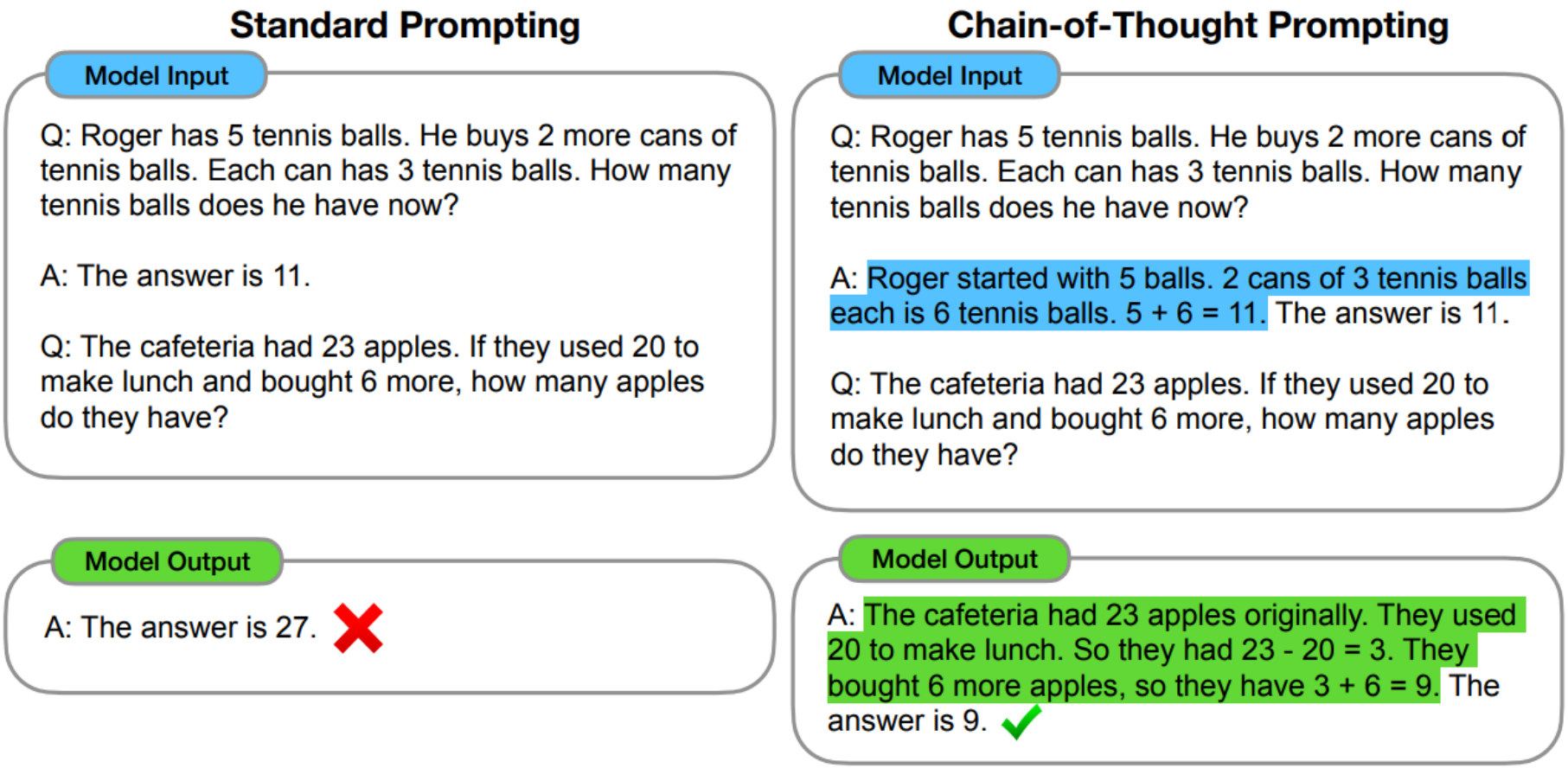
LLM Hallucinations

- LLMs can generate fluent and confident text that is factually incorrect: this is known as *hallucination*
- Two main types:
  - **Intrinsic**: contradicts the provided source or context
  - **Extrinsic**: introduces information not grounded in any source, which may be true or false
- Root causes:
  - Training data noise or gaps
  - Models optimized for fluency, not factuality
  - Knowledge cutoff: models ignore events after training
- Mitigation strategies: RAG, fine-tuning on factual data, TruthfulQA-style evaluation (Lin et al. 2022)

LLM Prompting

- Prompting may have a deep impact in the quality of LLM answers
- In-Context-Learning (Brown et al. 2020) (ICL) provides input-output pairs aiming that the LLM learns to perform a particular task by analogy
- A variant of ICL is called chain of thought (Wei et al. 2023) (CoT): CoT extends ICL by providing not only input-output pairs but also intermediate reasoning steps leading to the originally sought answers

LLM Prompting: CoT Example



NOTE: Figure taken from (Wei et al. 2023)

Retrieval-Augmented Generation (RAG)

- LLMs have a fixed knowledge cutoff and may hallucinate when asked about facts not seen during training
- **Retrieval-Augmented Generation** (P. Lewis et al. 2020) (RAG) addresses this by combining an LLM with an external knowledge source
- RAG pipeline:
  - **Indexing**: documents are split into chunks and stored as embeddings in a vector database
  - **Retrieval**: given a query, the most relevant chunks are retrieved by similarity search
  - **Generation**: the retrieved chunks are passed as context to the LLM, which generates a grounded answer
- Key advantage: knowledge can be updated without model retraining

Model Explainability

- Model explainability is a topic of growing interest in the field of machine learning
- Neural networks are black boxes
- When a prediction is made, it would be useful to know which sample features were relevant in the model decision
  - Better explainability of the model can make the user trust it more
  - Explainability information can also be used to troubleshoot the model predictions (real example: an image recognition system learned to classify images as horses by looking at a copyright tag that was exclusive to horse pictures, the problem was diagnosed by applying explainability techniques)

Model Explainability

- Two popular explainability techniques:
  - Shapley values (Shapley 1953)
  - Integrated gradients (Sundararajan et al. 2017)

- Main idea: calculate the contribution (or importance) of each feature to the network output for a particular sample

Model Explainability

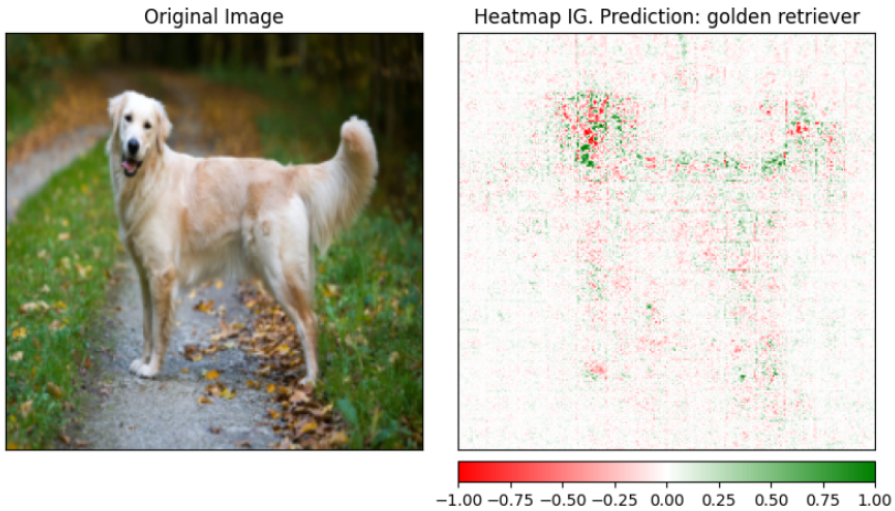
- Shapley values are more general and can be applied to any model, but they are computationally costly

- Integrated gradients can be applied to differentiable models only (any model trained with gradient descent, one example of non-differentiable model would be random forests), but they are faster to compute

- Implementations:
  - Shapley values: SHAP
  - Integrated gradients: Captum, Transformer Interpret

Model Explainability: Integrated Gradients Examples

- Example with images:



- Example with text:

Legend: Negative Neutral Positive

True Label	Predicted Label	Attribution Label	Attribution Score	Word Importance
POSITIVE	POSITIVE (1.00)	POSITIVE	2.02	[CLS] i love you , i like you [SEP]

LLM Agents

- An *LLM agent* is a system that uses an LLM as its core reasoning engine and can autonomously take actions to achieve a goal
- Key components:
  - **Planning**: decompose a complex task into sub-tasks (e.g., using CoT)
  - **Memory**: short-term (context window) and long-term (external storage or retrieval)
  - **Tools**: the agent can call external tools such as search engines, code interpreters, APIs, or databases
- Agents can operate in a loop: *observe* $\rightarrow$ *think* $\rightarrow$ *act* $\rightarrow$ *observe* ...
- Examples: OpenAI Assistants, LangChain agents, AutoGPT

References i

- Alon, Uri, Meital Zilberstein, Omer Levy, and Eran Yahav (2018). ***code2vec: Learning Distributed Representations of Code***. arXiv: 1803.09473 [cs.LG].
- Ao, Junyi, Rui Wang, Long Zhou, Chengyi Wang, Shuo Ren, Yu Wu, Shujie Liu, Tom Ko, Qing Li, Yu Zhang, Zhihua Wei, Yao Qian, Jinyu Li, and Furu Wei (2022). ***SpeechT5: Unified-Modal Encoder-Decoder Pre-Training for Spoken Language Processing***. arXiv: 2110.07205 [eess.AS].
- Bertasius, Gedas, Heng Wang, and Lorenzo Torresani (2021). ***Is Space-Time Attention All You Need for Video Understanding?*** arXiv: 2102.05095 [cs.CV].

References ii

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei (2020). ***Language Models are Few-Shot Learners***. arXiv: 2005.14165 [cs.CL].

References iii


Chang, Yupeng, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie (2023). ***A Survey on Evaluation of Large Language Models***. arXiv: 2307.03109 [cs.CL].


Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer (2023). ***QLoRA: Efficient Finetuning of Quantized LLMs***. arXiv: 2305.14314 [cs.LG].

References iv


Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). **"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding".** In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*.


Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby (2021). ***An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.*** arXiv: 2010.11929 [cs.CV].

References v

- Guo, Zishan, Renren Jin, Chuang Liu, Yufei Huang, Dan Shi, Supryadi, Linhao Yu, Yan Liu, Jiaxuan Li, Bojian Xiong, and Deyi Xiong (2023). ***Evaluating Large Language Models: A Comprehensive Survey***. arXiv: 2310.19736 [cs.CL].
- Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen (2021). ***LoRA: Low-Rank Adaptation of Large Language Models***. arXiv: 2106.09685 [cs.CL].

References vi

- Ji, Yanrong, Zhihan Zhou, Han Liu, and Ramana V Davuluri (Feb. 2021). **"DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome".** In: *Bioinformatics* 37.15, pp. 2112–2120. ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btab083. URL: https://doi.org/10.1093/bioinformatics/btab083.
- Khan, Salman, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah (Jan. 2022). **"Transformers in Vision: A Survey".** In: *ACM Computing Surveys* 54.10s, pp. 1–41. ISSN: 1557-7341. DOI: 10.1145/3505244. URL: http://dx.doi.org/10.1145/3505244.

References vii

- Latif, Siddique, Aun Zaidi, Heriberto Cuayahuitl, Fahad Shamshad, Moazzam Shoukat, and Junaid Qadir (2023). ***Transformers in Speech Processing: A Survey.*** arXiv: 2303.11607 [cs.CL].
- Lewis, Mike, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer (2019). ***BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.*** arXiv: 1910.13461 [cs.CL].
- Lewis, Patrick et al. (2020). **"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks".** In: *NeurIPS*.
- Li, Raymond, Loubna Ben Allal, and Yangtian Zi et. al (2023). ***StarCoder: may the source be with you!*** arXiv: 2305.06161 [cs.CL].

References viii

- Lin, Stephanie, Jacob Hilton, and Owain Evans (2022). **"TruthfulQA: Measuring How Models Mimic Human Falsehoods".** In.
- Liu, Yinhan, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov (2019). ***RoBERTa: A Robustly Optimized BERT Pretraining Approach.*** cite arxiv:1907.11692. URL: http://arxiv.org/abs/1907.11692.
- Luo, Ziyang, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang (2023). **"WizardCoder: Empowering Code Large Language Models with Evol-Instruct".** In: *arXiv preprint arXiv:2306.08568*.
- Ouyang, Long et al. (2022). **"Training language models to follow instructions with human feedback".** In: *NeurIPS*.

- Radford, Alec, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever (2018). **"Improving language understanding by generative pre-training".** In.
- Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever (2018). **"Language Models are Unsupervised Multitask Learners".** In: URL: https://d4mucfpksywv.cloudfront.net/better-language-models/language-models.pdf.
- Raffel, Colin, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu (2023). ***Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.*** arXiv: 1910.10683 [cs.LG].

References x

- Reimers, Nils and Iryna Gurevych (2019). ***Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks***. arXiv: 1908.10084 [cs.CL].
- Sanh, Victor, Lysandre Debut, Julien Chaumond, and Thomas Wolf (Oct. 2019). ***DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter***.
- Selva, Javier, Anders S. Johansen, Sergio Escalera, Kamal Nasrollahi, Thomas B. Moeslund, and Albert Clapes (2023). "**Video Transformers: A Survey**". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–20. ISSN: 1939-3539. DOI: 10.1109/tpami.2023.3243465. URL: http://dx.doi.org/10.1109/TPAMI.2023.3243465.

- Sennrich, Rico, Barry Haddow, and Alexandra Birch (Aug. 2016). **"Neural Machine Translation of Rare Words with Subword Units".** In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Katrin Erk and Noah A. Smith. Berlin, Germany: Association for Computational Linguistics, pp. 1715–1725. DOI: 10.18653/v1/P16-1162. URL: https://aclanthology.org/P16-1162.
- Shapley, Lloyd S (1953). **"A Value for n-Person Games".** In: *Contributions to the Theory of Games II*. Ed. by Harold W. Kuhn and Albert W. Tucker. Princeton: Princeton University Press, pp. 307–317.
- Sundararajan, Mukund, Ankur Taly, and Qiqi Yan (2017). ***Axiomatic Attribution for Deep Networks.*** arXiv: 1703.01365 [cs.LG].


Touvron, Hugo, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample (2023). ***LLaMA: Open and Efficient Foundation Language Models***. arXiv: 2302.13971 [cs.CL].


Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin (2017). **"Attention is all you need"**. In: *Advances in Neural Information Processing Systems*, pp. 5998–6008.


Wang, Yue, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi (Nov. 2021). **"CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation".** In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, pp. 8696–8708. DOI: 10.18653/v1/2021.emnlp-main.685. URL: https://aclanthology.org/2021.emnlp-main.685.


Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou (2023). ***Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.*** arXiv: 2201.11903 [cs.CL].


Xu, Peng, Xiatian Zhu, and David A. Clifton (2023). ***Multimodal Learning with Transformers: A Survey***. arXiv: 2206.06488 [cs.CV].